快速入门多维表“开发”

💡 由于目前执行环境更换，接口返回的数组再进行 map, find，filter，forEach 的时候需要加上 Array.from，比较靠谱的做法是使用 for （let i = 0; i < records.length; i++） 来操作

比如： const data = Application.Record.GetRecords()

data 是一个从多维表接口返回的数据，里面的 array 再进行操作的时候需要

const realRecords = Array.from(data.records)

💡 快速入门多维表“开发”【应用注释版】

水友版本 - 更友善

例子 1：选中区域快速批量填值

TypeScript

```typescript
function main(){
  const activeView = Application.Selection.GetActiveView()

  const selectedRecords = Application.Selection.GetSelectionRecords()[0]
  console.log(selectedRecords)
  const time = getNowTime()
  const date = getNowDate()
  const records = []
  for (let i = 0; i < selectedRecords.length; i++) {
    const item = selectedRecords[i]
    records.push({
      id: item.id,
      fields: {
        "日期": date,
        "时间": time,
        "分类": 'B'
      }
    })
  }

  // 注意，这里的 “日期”， “时间”， “分类”， 需要替换到您表中的相应字段名
  Application.Record.UpdateRecords({
    SheetId: activeView.sheetId,
    Records: records,
  })
}
// 获取当前时间，格式为 "hh:mm:ss"
function getNowTime() {
  return (new Date()).toTimeString().split(" ")[0]
}
// 获取当前日期，格式为 "yyyy:MM:dd"
function getNowDate() {
  const date = new Date()
  return date.getFullYear() + '/' + (date.getMonth() + 1) + '/' + date.getDate()
}

main()
```

例子 2：快速实现“一键归档”

下面代码实现了一个文件中两张数据结构相同的表

把表一中的已完成的数据插入到表二中，并删除表一中数据

表结构如下图所示：

TypeScript

```typescript
function main() {
  const sheets = Application.Sheet.GetSheets()
  // 筛选出 打钩的 并且是分类 B 中的数据
  const finishedRecords = Application.Record.GetRecords({
    SheetId: sheets[0].id,
    Filter: {
      mode: 'AND',
      criteria: [
        {
          field: '完成',
          op: 'Equals',
          values: ['1'],
        },
        {
          field: '分类',
          op: 'Equals',
          values: ['B']
        }
      ]
    }
  })
  // 如果存在 筛选出来的数据
  if (finishedRecords) {
    // 在表二中插入数据
    Application.Record.CreateRecords({
      SheetId: sheets[1].id,
      Records: Array.from(finishedRecords.records).map(item => ({
        fields: item.fields,
      }))
    })
    // 在表一中删除数据
    Application.Record.DeleteRecords({
      SheetId: sheets[0].id,
      RecordIds: Array.from(finishedRecords.records).map(item => item.id),
    })
  }
}
main()
```

📌 结合上面两个例子，可以实现自动设置归档日期和时间，或者选中记录一键归档等等功能

例子 3：快速删除空白数据

JavaScript

```javascript
function fetchAllRecords() {
  const view = Application.Selection.GetActiveView()
  let all = []
  let offset = null;

  while (all.length === 0 || offset) {
    let records = Application.Record.GetRecords({
      SheetId: view.sheetId,
      ViewId: view.viewId,
      Offset: offset,
    })
    offset = records.offset
    all = all.concat(records.records)
  }
  console.log(all.length)
  return all
}

function deleteRecords() {
  const view = Application.Selection.GetActiveView()
  const records = fetchAllRecords()
  // 删除名称字段中 值为空的数据
  let recordIds = records.filter(it => !it.fields['名称'])
  recordIds = recordIds.map(item => item.id)
  console.log(recordIds)

  Application.Record.DeleteRecords({
    SheetId: view.sheetId,
    RecordIds: recordIds,
  })
}

deleteRecords()
```

例子 4：快速创建一张表

TypeScript

```typescript
function main(){
  Application.Sheet.CreateSheet({
    Name: "我的表",
    Fields: [
      { name: '名称', type: 'MultiLineText' },
      { name: '数量', type: 'Number' },
      { name: '日期', type: 'Date' },
      { name: '时间', type: 'Time' },
      { name: '复选框', type: 'Checkbox' },
      { name: '超链接', type: 'Url' },
      { name: '等级', type: 'Rating', max: 5},
      { name: '电话', type: 'Phone' },
      { name: '身份证', type: 'ID'},
      { name: '货币', type: 'Currency' },
      { name: '百分比', type: 'Percentage'},
      { name: '邮箱', type: 'Email'},
      { name: '进度', type: 'Complete'},
      { name: '分类', type: 'SingleSelect',
        items: [{ value: 'A' }, { value: 'B' }, { value: 'C'} ]
      },
      { name: '状态', type: 'MultipleSelect',
        items: [{ value: '已完成' }, { value: '未开始'}, { value: '进行中' }]
      },
      { name: '联系人',type: 'Contact',
        multipleContacts: false, noticeNewContact: false,
      },
      { name: '富文本', type: 'Note', },
      { name: '附件', type: 'Attachment'},
      { name: '公式', type: 'Formula' },
      { name: '创建时间', type: 'CreatedTime' },
      { name: '创建者', type: 'CreatedBy' },
      { name: 'AutoNumber', type: 'AutoNumber' }
    ],
    Views: [
      { name: '表格', type: 'Grid' },
      { name: '看板', type: "Kanban" },
      { name: '画册', type: 'Gallery'},
      { name: '表单', type: 'Form' },
      { name: '甘特', type: 'Gantt'}
    ],
  })
```

```
}
main()

```

例子 5：格式化数据批量插入

TypeScript

```
function main(){
  const template = ["商品", 10]
  const records = []
  // 插入 100 条 A 类商品，
  for (let i = 1; i < 100; i++ ) {
    records.push({
      "名称": template[0] + i,
      "数量": template[1],
      "分类": 'A'
    })
  }
  for (let i = 1; i < 100; i++ ) {
    records.push({
      "名称": template[0] + (100 + i),
      "数量": template[1] + 10,
      "分类": 'B'
    })
  }
  for (let i = 1; i < 100; i++ ) {
    records.push({
      "名称": template[0] + (200 + i),
      "数量": template[1] + 10,
      "分类": 'C'
    })
  }
  const sheet = Application.Selection.GetActiveSheet()
  Application.Record.CreateRecords({
    SheetId: sheet.sheetId,
    Records: records.map(item => ({
      fields: item,
    })),
  })
}
main()
```

例子 6：自动双向关联

先来看一下表结构

- 客户表

	客户名称	对接人姓名	重要程度	拜访记录	+
1	金山办公	张三	S级		
2	猎豹移动	李四	A级		
3	西山居	王五	B级		
+					

- 拜访记录表

	客户名称	拜访日期	拜访内容	客户详情	+
1	金山办公	2022/10/08	测试1		
2	猎豹移动	2022/10/09	测试2		
3	西山居	2022/10/15	测试3		
4	金山办公	2022/10/01	测试4		
5	金山办公	2022/10/07	测试5		
	西山居	2022/10/07	测试6		
+					

对 【客户表】 中的"拜访记录" 和 【拜访记录表】 中的 "客户详情"做自动关联

> 注：以下两个表的 id，是通过 Application.Sheet.GetSheets() 查询得到的

JavaScript

```javascript

function main() {
  const visits = getAllRecordsFromVisitSheet()
  const clients = getAllRecordsFromClientSheet()
  linkVisits(clients, visits)
  linkClients(clients, visits)
}

// 关联拜访记录表中的客户
function linkClients(clients, visits) {
  const allClients = formClientStore(clients)
  let result = visits.map(item => {
    const name = item.fields['客户名称']
    const relateClient = allClients[name]
    if (!relateClient) {
      return null
    }
    return {
      id: item.id,
      fields: {
        "客户详情": {
          recordIds: [relateClient.id]
        }
      }
    }
  })
  result = result.filter(item => !!item)
  console.log(result)
  Application.Record.UpdateRecords({
    SheetId: 3,
    Records: result,
  })
}


// 关联客户表中的 拜访记录
function linkVisits(clients, visits) {
  const allClients = formClientStore(clients)
  // 遍历将记录逐条加入 刚才的对象中
  for (let i = 0; i < visits.length; i++) {
    const name = visits[i].fields["客户名称"]
    if (allClients[name]) {
```

```
      allClients[name].recordIds.push(visits[i].id)
    }
  }
  // 更新记录
  const keys = Object.keys(allClients)
  Application.Record.UpdateRecords({
    SheetId: 1,
    Records: keys.map(key => {
      const item = allClients[key]
      return {
        id: item.id,
        fields: {
          "拜访记录": {
            recordIds: item.recordIds,
          }
        }
      }
    }),
  })
}

// 初始化一个保存所有客户数据的对象
function formClientStore(clients) {
  const store = {}
  for (let i = 0; i < clients.length; i++) {
    let name = clients[i].fields["客户名称"]
    store[name] = {
      id: clients[i].id,
      recordIds: []
    }
  }
  return store
}


// 从客户详情表读取所有记录 sheetId 固定为 1
function getAllRecordsFromClientSheet() {
  return getAllRecords(1)
}

// 从拜访记录表中读取所有记录 sheetId 固定为 3
function getAllRecordsFromVisitSheet() {
  return getAllRecords(3)
}
```

```
// 读取整个表
function getAllRecords(SheetId) {
  let all = []
  let offset = null;

  while (all.length === 0 || offset) {
    let records = Application.Record.GetRecords({
      SheetId,
      Offset: offset,
    })
    offset = records.offset
    all = all.concat(records.records)
  }
  return all
}

main()
```

例子 7：同步主表数据到其他表

表结构如下

> 第一张表中存着所有数据，其他几张表中存着分类数据

> 当主表中数据更新的时候，期望其他表数据也更新

JavaScript

```javascript
// 读取整个表
function getAllRecords(SheetId) {
  let all = []
  let offset = null;

  while (all.length === 0 || offset) {
    let records = Application.Record.GetRecords({
      SheetId,
      Offset: offset,
    })
    offset = records.offset
    all = all.concat(records.records)
  }
  return all
}

// 检查记录值相同与否，返回需要更新的记录
function checkFields(mainRecord, record, params) {
  const mainFields = mainRecord.fields
  const fields = record.fields
  const diffFields = {}
  for (let i = 0; i < params.length; i++) {
    const param = params[i]
    if (mainFields[param] !== fields[param]) {
      diffFields[param] = mainFields[param]
    }
  }
  if (Object.keys(diffFields).length > 0) {
    console.log(diffFields)
    return {
      id: record.id,
      fields: diffFields,
    }
  }
  return null
}

// 同步表数据
function syncSheet(SheetId, mainRecordMap, {
  keyField,
  toSyncFields,
}) {
```

```
  const records = getAllRecords(SheetId)
  const toUpdateRecords = []
  for (let i = 0; i < records.length; i++) {
    const record = records[i]
    const name = record.fields[keyField]
    const mainRecord = mainRecordMap[name]
    if (mainRecord) {
      const updatedRecord = checkFields(mainRecord, record, toSyncFields)
      if (updatedRecord) {
        toUpdateRecords.push(updatedRecord)
      }
    } else {
      console.error('没有在主表里面找到此条记录:  ', name)
    }
  }
  console.log(toUpdateRecords)
  if (toUpdateRecords.length > 0) {
    Application.Record.UpdateRecords({
      SheetId,
      Records: toUpdateRecords,
    })
  }
}

// 同步主表数据到其他表
function syncMainSheetToOthers(mainSheetName, keyField, toSyncFields) {
  const sheets = Application.Sheet.GetSheets()
  const mainIndex = sheets.findIndex(item => item.name === mainSheetName)
  const mainSheet = sheets[mainIndex]
  const mainRecords = getAllRecords(mainSheet.id)
  const recordsMap = {}
  for (let i = 0; i < mainRecords.length; i++) {
    const name = mainRecords[i].fields[keyField]
    if (recordsMap[name]) {
      console.error('有重复的记录', name)
    }
    recordsMap[name] = mainRecords[i]
  }

  sheets.splice(mainIndex, 1)

  sheets.forEach(sheet => {
```

```
    syncSheet(sheet.id, recordsMap, {
      keyField,
      toSyncFields,
    })
  })
}

syncMainSheetToOthers('😍关爱清单', '产品名', ['机制', '价格'])
```

例子 8：获取日期筛选后记录

表结构如下

	客户名称	日期	拜访内容
1	金山办公	2023/01/05	测试1
2	猎豹移动	2023/01/26	测试2
3	西山居	2023/01/13	测试3
	金山办公	2023/01/05	测试1
5	猎豹移动	2023/02/08	测试2
6	西山居	2023/02/17	测试3
7	金山办公	2023/02/08	测试1
8	猎豹移动	2023/02/04	测试2
9	西山居	2023/02/04	测试3

使用日期筛选进行获取日期为本月（2023 年 2 月）的记录，即将筛选参数改为 thisMonth，如"dynamicType": "thisMonth", 筛选得到的记录可查看 all 变量

JavaScript

```javascript

function filterRecord (){
  const view = Application.Selection.GetActiveView()
  let all = []
  let offset = null;

  while (all.length === 0 || offset) {
    let records = Application.Record.GetRecords({
      SheetId: view.sheetId,
      ViewId: view.viewId,
      Offset: offset,
      Filter: {
        "mode": "AND",
            "criteria": [
                {
                    "field": "日期",
                    "op": "Equals",
                    "values": [
                        {
                            "dynamicType": "thisMonth",
                            "type": "DynamicSimple"
                        }
                    ]
                }
            ]
      }
    })
    offset = records.offset
    all = all.concat(records.records)
  }
  console.log(all.length)
  console.log(all)
  return all

}

filterRecord ();

```

结果如下：所获取记录的日期全为本月（2023 年 2 月）

```
▼[[PromiseResult]]: Array(5)
  ▼0:
    ▶fields: {关联: 客户表: Array(1), 客户名称: '猎豹移动', 拜访内容: '测试2', 日期: '2023/02/08'}
     id: "BHS"
    ▶[[Prototype]]: Object
  ▼1:
    ▶fields: {关联: 客户表: Array(1), 客户名称: '西山居', 拜访内容: '测试3', 日期: '2023/02/17'}
     id: "BHT"
    ▶[[Prototype]]: Object
  ▼2:
    ▶fields: {关联: 客户表: Array(1), 客户名称: '金山办公', 拜访内容: '测试1', 日期: '2023/02/08'}
     id: "BHU"
    ▶[[Prototype]]: Object
  ▼3:
    ▶fields: {关联: 客户表: Array(1), 客户名称: '猎豹移动', 拜访内容: '测试2', 日期: '2023/02/04'}
     id: "BHV"
    ▶[[Prototype]]: Object
  ▼4:
    ▶fields: {关联: 客户表: Array(1), 客户名称: '西山居', 拜访内容: '测试3', 日期: '2023/02/04'}
     id: "BHW"
    ▶[[Prototype]]: Object
   length: 5
```

例子 9：获取联系人筛选后记录

表结构如下

客户名称	日期	拜访内容	联系人
金山办公	2023/01/05	测试1	Crush
猎豹移动	2023/01/26	测试2	Lin
西山居	2023/01/13	测试3	Lin Crush
金山办公	2023/01/05	测试1	Lin
猎豹移动	2023/02/08	测试2	Crush
西山居	2023/02/17	测试3	Lin
金山办公	2023/02/08	测试1	Lin
猎豹移动	2023/02/04	测试2	Crush

使用联系人筛选进行获取联系人为 Lin 的记录，即更改筛选参数 value 改为“Lin”，如 "values": ["Lin"]，注意 value 值为数组形式，筛选得到的记录可查看 all 变量

JavaScript

```javascript
function filterContact() {
  const view = Application.Selection.GetActiveView()
  let all = []
  let offset = null;

  while (all.length === 0 || offset) {
    let records = Application.Record.GetRecords({
      SheetId: view.sheetId,
      ViewId: view.viewId,
      Offset: offset,
      Filter: {
        "mode": "AND",
            "criteria": [
                {
                    "field": "联系人",
                    "op": "Equals",
                    "values": [
                          "Lin"
                    ]
                }
            ]
      }
    })
    offset = records.offset
    all = all.concat(records.records)
  }
  console.log(all.length)
  console.log(all)
  return all

}

filterContact()
```

例子 10：批量将获取的图片 url 进行文字识别

名称	数量	日期	时间	分类	关联	图片和附件	识别文字
冰箱		2023/02/20	17:42	C			
音响		2023/01/13	17:42	C	音响		{"words_result":[{"words":"at iterateNorm...
空调		2023/01/12	17:42	C	空调		{"words_result":[{"words":"at iterateNorm...
冰箱		2022/12/07	17:42	C	冰箱		{"words_result":[{"words":"at iterateNorm...
电视		2023/02/24					
沙发		2023/02/24					{"words_result":[{"words":"at iterateNorm...

- 可以通过获取的图片 url 来进行调用百度云文字识别接口，识别图片中的文字

获取所有记录，将字段类型为 attachment 的字段名称传入 `fetchAllAttachments(fieldsName)` 获取该表所有图片 URL，调用百度云文字识别接口，识别图片中的文字，并将结果生成在相应记录的“识别文字”字段下。

JavaScript

```javascript
const access_token="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

function fetchAllRecords() {
  const view = Application.Selection.GetActiveView()
  let all = []
  let offset = null;

  while (all.length === 0 || offset) {
    let records = Application.Record.GetRecords({
      SheetId: view.sheetId,
      ViewId: view.viewId,
      Offset: offset,
    })
    offset = records.offset
    all = all.concat(records.records)
  }
  console.log(all.length)
  return all
}

function fetchAllAttachments(fieldsName){
  const records = fetchAllRecords()
  const urls = records.map((item) => {
    if(item.fields[fieldsName] === undefined) return [item.id,null]
    else {
      const source = item.fields[fieldsName][0].source
      const uploadId = item.fields[fieldsName][0].uploadId
      const url = Application.Record.GetAttachmentURL({
          UploadId: uploadId,
          Source: source
      })
      return [item.id, url]
    }
  })
  console.log(urls)
  return urls
}


function recognition(){
  const urls = fetchAllAttachments('图片和附件')
  const words_result = [];
```

```
  const sheetId = Application.Selection.GetActiveView().sheetId
  console.log(sheetId)
  if(!Application.Field.GetFields({ SheetId: sheetId}).find(item => {
    item.name === '识别文字'
  })) {
    Application.Field.CreateFields({
      SheetId: sheetId,
      Fields: [
          { name: '识别文字',  type: 'MultiLineText' }
      ]
    })
  }
  for (i in urls){
    if(urls[i][1] === null) continue
    console.log(urls[i][1])
    let resp = HTTP.fetch('https://aip.baidubce.com/rest/2.0/ocr/v1/general_basic?access_token='+access_token,{
    method:"POST",
    timeout:2000,
    headers:{
      'Content-Type': 'application/x-www-form-urlencoded',
      'Accept': 'application/json'
    },
    body: `url=${encodeURIComponent(urls[i][1])}`
    })
    //console.log(resp.status)
    if(resp.status() !== 200){
    throw new Error("fetch err! status is "+resp.status())
    }

     Application.Record.UpdateRecords({
        SheetId: sheetId,
        Records: [{
            id: urls[i][0],
            fields: {
                 "识别文字": resp.text(),
            }
        }],
    })
  }
  //console.log( words_result)
  return words_result;
}
recognition()
```